

Voice Control Color Sorting

1. Function Description

Through voice commands, give the block sorting order, and the robotic arm will sort the blocks according to the given order.

This case requires flipping the map to the side with the cross symbol.

2. Startup and Operation

2.1 Startup

First, start the ROS node service. Open the mainboard terminal and enter the following:

```
ros2 run dofbot_info kinemarics_dofbot
```

Then enter the following command in the terminal to start:

```
python3 ~/dofbot_voice/scripts/voice_color_sorting.py
```

Finally, open the terminal and enter the following command to start:

2.2 Operation Steps

After the program runs, the voice broadcast module will broadcast the colors of all blocks in the frame. For example, in the figure below, it will broadcast the recognized colors, such as "I see red, blue, yellow, green".

Note: Color recognition effect varies with ambient light, and can only sort recognized colors.



At this time, wake up the voice module with the keyword "Hi, yahboom", and say the first color to sort, for example "Red". The program will record the first color to sort. The program image will show the added color. Then you can say the next color, for example "Green". If you don't want to sort a certain color, you can say "No". Then the block that was just added to the sorting list will be

removed from the queue. After completing the sorting, you can say "Start sorting", it will broadcast "ok", and then execute the sorting action.

Note: If the color cannot be recognized, you may need to modify the HSV values in the program or adjust the ambient light.

3. Core Code Analysis

Source code path: ~/dofbot_voice/scripts/voice_color_sorting.py

```
# Mainly look at the imported library files, speech_identify_target library,
which is located at ~/dofbot_pro/dofbot_color_sorting/scripts/identify_target.py
from speech_identify_target import identify_GetTarget
## Create get target instance
target = identify_GetTarget()
# Call select_color function to select, determine the color and block position
img, msg, coo1, coo2, coo3, coo4 = target.select_color function, determine color
and block position(img, color_hsv, color_list)
# Start thread, launch grabbing program
threading.Thread(target=target.target_run, args=(msg,xy)).start()
```

In the speech_identify_target library,

```
# Select recognition color, the returned msg includes the selected color and the
block position
def select_color(self, image, color_hsv, color_list):
    '''
    Choose a recognition color
    选择识别颜色
    :param image: input image
    :param color_hsv: Range threshold for HSV
    :param color_list: Color sequence: ['0': None '1': Red '2': Green '3': Blue
    '4': Yellow]
    :return: Output the processed image, (color, position)
    '''
    # Canonical input image size
    self.image = image.copy()
    msg = {}
    # Get information about the first color sequence
    if len(color_list)==0: return self.image, msg
    if '1' in color_list:
        self.color_name=color_list['1']
        pos = self.get_Sqaure(color_hsv[self.color_name])
        if pos != None: msg[self.color_name] = pos
    if '2' in color_list:
        self.color_name=color_list['2']
        pos = self.get_Sqaure(color_hsv[self.color_name])
        if pos != None: msg[self.color_name] = pos
    if '3' in color_list:
        self.color_name=color_list['3']
        pos = self.get_Sqaure(color_hsv[self.color_name])
        if pos != None: msg[self.color_name] = pos
    if '4' in color_list:
        self.color_name=color_list['4']
        pos = self.get_Sqaure(color_hsv[self.color_name])
        if pos != None: msg[self.color_name] = pos
```

```

        return self.image, msg

# Grab function
def target_run(self, msg, xy=None):
    """
    grab function
    :param msg: (color, position)
    """
    if xy != None: self.xy = xy
    num = 1
    move_status=0
    for i in msg.values():
        if i !=None: move_status=1
    if move_status==1:
        self.arm.Arm_Buzzer_On(1)
        sleep(0.5)
    for name, pos in msg.items():
        # Here, ROS inversely solves the communication to obtain the rotation
        angle of each joint
        joints = self.server_joint(pos)
        self.grap.identify_move(str(name), joints)
        num += 1

    if move_status==1:
        # set up
        joints_uu = [90, 80, 50, 50, 90, 30]
        # Move over the object's position
        self.arm.Arm_serial_servo_write6_array(joints_uu, 1000)
        sleep(1.5)
        # initial position
        joints_0 = [self.xy[0], self.xy[1], 0, 0, 90, 30]
        # move to initial position
        self.arm.Arm_serial_servo_write6_array(joints_0, 1000)
        sleep(1.5)

# Call inverse kinematics service, provide xy to calculate the rotation angle of
each joint
# If the clamping effect is not satisfactory, you can modify the request.tar_x,
request.tar_y, and request.tar_z below here appropriately by adding or
subtracting them
def server_joint(self, posxy):
    """
    Post position request, get joint rotation angle
    :param posxy: Location point x,y coordinates
    :return: Rotation angle of each joint
    """

    # Create a message pack
    request = Kinemarics.Request()
    request.tar_x = posxy[0]
    request.tar_y = posxy[1]
    request.tar_z = 0.02
    request.roll = -1.57
    request.pitch = 0.0
    request.yaw = 0.0
    request.kin_name = "ik"
    future = self.client.call_async(request)
    rclpy.spin_until_future_complete(self, future)
    response = future.result()
    if response is not None:

```

```
# Get the inverse solution response result
joints = [0.0, 0.0, 0.0, 0.0, 0.0]
joints[0] = response.joint1
joints[1] = response.joint2
joints[2] = response.joint3
joints[3] = response.joint4
# joints[4] = response.joint5
joints[4] = posxy[2] + 90
return joints
else:
    self.get_logger().info('Service call failed')
```